



Projet de Fin d'Études : Majeure Ingénierie Financière et Statistiques

DEEP LEARNING ET RÉSEAUX DE NEURONES APPLIQUÉS À LA FINANCE

“Application de modèles de réseaux de neurones pour la prédiction de cours de sous-jacents afin d’optimiser l’allocation d’actifs d’un portefeuille ”

ARMAND Mattéo, HARDY Jade, SAMAMA Noam

Supervisé par M. Louhmedi

2025

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
1. Introduction générale.....	4
1.1. Contexte du projet	4
1.2. Motivations.....	4
1.3. Structure du document	4
2. Partie technique théorique.....	5
2.1. Structure générale commune	5
2.2. Section ANN	7
2.2.1 Rôle du réseau de neurones artificiel (ANN) dans notre projet	7
2.2.2 Modélisation mathématique d'un ANN	7
2.2.3 Données financières et construction du jeu d'apprentissage	9
2.2.4 De la prédiction individuelle à l'allocation de portefeuille.....	10
2.2.5 Protocole d'entraînement et validation (walk-forward)	11
2.2.6 Implémentation Python : ANN + allocation	12
2.2.7 Limites de l'ANN et intérêt pour la suite du projet	13
2.3. Partie CNN.....	14
2.3.1 Motivation et positionnement des CNN dans le projet	14
2.3.2 Principes mathématiques du CNN.....	15
2.3.3 Données et financières et représentation en entrée du CNN	19
2.3.4 Limite des CNN pour la suite du projet.....	20
2.4. Réseaux récurrents (RNN) pour séries temporelles financières	22
2.4.1 Motivation des RNN et Séries temporelles financières	22
2.4.2 Prétraitement des séries temporelles financières pour les RNN	23
2.4.3 Formulation d'un RNN "vanilla"	24
2.4.4 Différentes formes d'entrées/sorties (many-to-one, many-to-many).....	27
2.4.5 Fonction de perte et entraînement (BPTT).....	28
2.4.6 Problème du gradient évanescent / explosif	29
2.4.7 Limitations pratiques des RNN vanilla et motivation pour GRU / LSTM	30
2.4.8 Application typique d'un RNN vanilla à notre projet (pour comparaison)	30
2.5. Long Short-Term Memory (LSTM)	31
2.5.1 Origine et motivation du modèle LSTM	31
2.5.2 Structure générale d'une cellule LSTM	31
2.5.3 Formulation mathématique complète du LSTM	32
2.5.4 Schéma conceptuel de la cellule LSTM	33
2.5.5 Intuition dynamique et stabilité de l'apprentissage	33
2.5.6 Interprétation financière du LSTM	34
2.5.7 LSTM pour la prédiction financière dans notre projet	34
2.5.8 LSTM et allocation de portefeuille	35
2.5.9 Limites du LSTM.....	35

3. D�veloppement et interface utilisateur.....	38
3.1. Objectif du site	38
3.2. Structure g�n�rale de l'interface	38
3.3. Barre lat�rale gauche (ruban).....	38
3.3.1. Section « Importation des donn�es ».....	38
3.3.2. Section « Param�tres d'entra�nement »	39
3.3.3. Section « Pr�vision future »	39
3.3.4. Section « Actions »	39
3.4. Zone principale droite	40
3.4.1. Navigation	40
3.4.2. Onglet 1 – « Visualisation des pr�visions »	40
3.4.3. Onglet 2 – « Allocation du portefeuille »	41
3.5. Comportements / logiques de base	41

1. INTRODUCTION GENERALE

1.1. Contexte du projet

Le présent projet consiste à étudier, concevoir et implémenter plusieurs modèles de réseaux de neurones afin de les appliquer à deux problématiques financières majeures :

- La prédiction de cours d'actifs financiers.
- La complétion de séries temporelles présentant des données manquantes, problématique fréquente dans les bases de données financières réelles.

Les séries temporelles constituent le socle de la prédiction des cours d'actifs, car elles permettent de modéliser l'évolution d'un prix en tenant compte de sa dynamique passée. Les observations successives (prix, rendements, volatilité) sont analysées comme des données dépendantes dans le temps, afin d'identifier des tendances, des comportements récurrents et des structures de dépendance exploitables par des modèles statistiques ou de machine learning. L'objectif est de capter l'information contenue dans l'historique des cours pour anticiper leurs mouvements futurs, tout en intégrant l'incertitude et le caractère aléatoire des marchés financiers.

L'objectif final est de développer un outil Python intégrant une interface utilisateur, permettant de :

- Charger des données financières,
- Entraîner différents modèles (ANN, CNN, RNN),
- Visualiser leurs prédictions,
- Vérifier leur cohérence via un backtest historique,
- Proposer une allocation de portefeuille basée sur ces prédictions.

1.2. Motivations

Les réseaux de neurones offrent une approche flexible et non linéaire adaptée aux séries financières, souvent bruitées, incomplètes et non stationnaires. Ce projet doit démontrer à la fois notre compréhension théorique des architectures neurales, notre capacité à traduire ces concepts en code, et la rigueur dans la modélisation de données financières.

1.3. Structure du document

Le document est organisé en trois grandes parties :

- Partie technique théorique : description détaillée des modèles ANN, CNN, RNN/LSTM/GRU.
- Partie technico-fonctionnelle : données, prétraitements, choix méthodologiques, interface, pipeline.

- Partie opérationnelle : méthodologie d'implémentation, tests, backtests, allocation de portefeuille.

2. PARTIE TECHNIQUE THEORIQUE

Cette partie consiste principalement à fournir une description détaillée des différents modèles de réseaux de neurones que nous avons rencontré et que nous saurions près à utiliser dans le cadre de ce projet.

2.1. Structure générale commune

Les données de marché comportent souvent des erreurs, doublons, valeurs aberrantes ou manquantes résultant de problèmes techniques, de suspensions de cotation, ou d'événements exceptionnels. Ces imperfections, si elles ne sont pas corrigées, peuvent fausser les apprentissages du modèle. Le nettoyage débute par une détection systématique des incohérences : recherche de valeurs manquantes, identification de doublons, vérification des horodatages et alignement des fréquences temporelles (journalière, horaire, etc.). La présence de valeurs aberrantes (par exemple un prix isolé très éloigné du reste de la distribution) peut être mise en évidence par des tests statistiques (z-score, écart interquartile) ou des visualisations (boîtes à moustaches). Selon leur origine, ces éléments peuvent être supprimés, transformés, ou bornés (capping/flooring) afin de réduire leur poids dans l'apprentissage. La complétion des séries temporelles constitue une étape critique, car la suppression directe des lignes incomplètes entraînerait une perte importante d'information. Plusieurs approches sont possibles :

- Imputation statistique : remplacement des valeurs manquantes par la moyenne, la médiane ou la dernière observation connue (méthode forward-fill), adaptée lorsque les manques sont rares et aléatoires.
- Imputation par proximité (KNN) : estimation des valeurs manquantes à partir d'observations similaires dans l'espace des variables.
- Imputation par modèles : recours à des modèles multivariés ou neuronaux (comme les autoencodeurs ou RNN) capables de reconstruire la dynamique manquante à partir du contexte temporel environnant.

Les variables financières présentent souvent des échelles de grandeur très différentes: un prix d'action peut valoir plusieurs centaines d'euros, tandis qu'un rendement journalier est exprimé en pourcentage. Ces écarts de magnitude peuvent perturber l'apprentissage des réseaux de neurones, car certaines variables dominent les autres dans le calcul des gradients. Deux techniques principales sont utilisées pour uniformiser les données :

- La normalisation (Min-Max Scaling), qui ramène toutes les variables dans un intervalle borné (souvent $[0,1]$). Elle est particulièrement adaptée aux réseaux de neurones, car elle favorise la stabilité et la convergence pendant l'entraînement.
- La standardisation (Z-score Scaling), qui centre les données autour de zéro et les réduit selon leur écart type. Cette approche est plus robuste lorsque les distributions comportent des valeurs extrêmes ou non bornées.

Ces transformations améliorent la performance et la vitesse de convergence des modèles tout en assurant que chaque variable contribue équitablement à la décision du réseau. L'hypothèse de stationnarité est centrale dans l'analyse des séries temporelles. Un modèle de Deep Learning appliqué à une série non stationnaire risque de confondre tendance et signal, ce qui réduit sa capacité de généralisation. En cas de non-stationnarité, plusieurs techniques peuvent être appliquées :

- La différenciation classique, qui consiste à soustraire chaque observation de la précédente pour supprimer les tendances.
- La différenciation logarithmique, utile lorsque la série croît de manière exponentielle.
- La différenciation fractionnaire (fractional differencing), introduite par Lopez de Prado, qui préserve partiellement la mémoire du processus tout en rendant la série stationnaire.

Cette méthode est particulièrement pertinente pour les séries financières, où les dépendances à long terme contiennent souvent des signaux prédictifs.

Une fois les données nettoyées et normalisées, elles doivent être transformées en séquences temporelles exploitables par les architectures neuronales récurrentes (RNN, LSTM, GRU). Cette étape, appelée *sliding window*, consiste à découper la série en sous-séquences glissantes d'une taille fixe (par exemple, 30 jours d'observations pour prédire le 31e). Ce format permet au modèle d'apprendre les dépendances temporelles locales et les dynamiques de court terme tout en conservant la structure séquentielle des données. Le prétraitement des données financières est donc une étape déterminante du pipeline de Deep Learning. Il combine rigueur statistique et choix méthodologiques adaptés à la nature particulière des marchés : bruit, volatilité, non-stationnarité et dépendances complexes. Dans notre approche centrée sur les actions, la préparation des données constitue le socle de tout apprentissage : un jeu de données propre, cohérent et bien structuré maximise non seulement la performance prédictive, mais surtout la fiabilité des conclusions économiques qui en découlent.

2.2. Section ANN

2.2.1 Rôle du réseau de neurones artificiel (ANN) dans notre projet

Dans notre projet de fin d'études, on cherche à utiliser le Deep Learning pour aider à la décision d'investissement sur un panier d'actions. L'idée n'est pas simplement de dire "cette action va monter", mais de construire, à chaque date, une allocation de portefeuille qui répartit le capital entre plusieurs titres.

Le rôle de l'ANN dans ce cadre est clair :

- Pour chaque action de l'univers, à une date t , on lui fournit un vecteur de caractéristiques financières (rendements passés, volatilité, indicateurs techniques simples).
- Le réseau prédit une quantité liée à la performance future (par exemple le log-rendement à horizon h jours).
- On convertit ensuite les prédictions pour toutes les actions de la date t en poids de portefeuille : chaque action reçoit une pondération $w_{i,t}$, et on a $\sum_i w_{i,t} = 1$.
- On backteste la stratégie obtenue pour évaluer si cette combinaison ANN + règle d'allocation apporte un gain de performance ou de gestion du risque par rapport à des benchmarks simples (égal-pondéré, buy&hold, etc.).

Concrètement, l'ANN transforme l'information passée en espérances de rendement, et c'est derrière cela qu'on injecte de la finance (gestion de portefeuille, contraintes, coûts de transaction).

2.2.2 Modélisation mathématique d'un ANN

2.2.2.1 Structure d'un réseau de neurones artificiel

On considère un réseau de neurones de type Multi-Layer Perceptron (MLP). Il prend en entrée un vecteur $x \in \mathbb{R}^d$ (les features d'une action à une date donnée) et renvoie une sortie scalaire $\hat{y}$ (le rendement futur prédit ou un score).

On note :

- $a^{(0)} = x$: les activations d'entrée,
- pour chaque couche $\ell = 1, \dots, L$:

matrice de poids $W^{(\ell)}$,

vecteur de biais $b^{(\ell)}$,

fonction d'activation non linéaire $\phi^{(\ell)}$.

La propagation avant se fait comme suit :

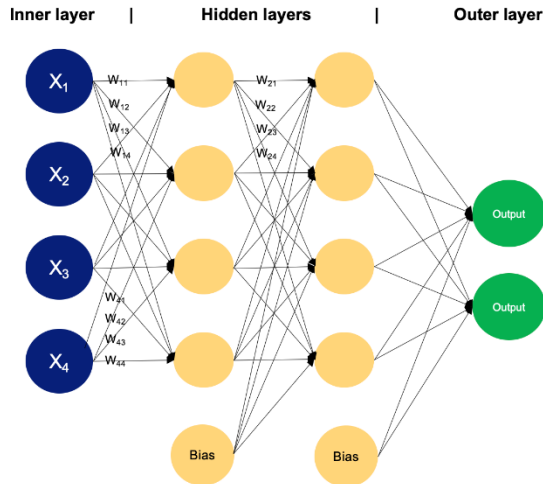
- $z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}$,
- $a^{(\ell)} = \phi^{(\ell)}(z^{(\ell)})$.

La sortie finale est :

- $\hat{y} = a^{(L)} = f_{\theta}(x)$,

où θ regroupe l'ensemble des paramètres du réseau (tous les $W^{(\ell)}$ et $b^{(\ell)}$).

Dans notre cas, f_{θ} représente une approximation non linéaire de la relation entre les caractéristiques d'un titre et son rendement futur.



Architecture du réseau de neurones artificiel (ANN)

2.2.2.2 Fonction de perte pour la prédiction de rendements

On traite d'abord le problème comme une régression : on veut prédire le log-rendement futur $y_{i,t} = r_{i,t+h}$ de l'action i à partir de son vecteur de features $x_{i,t}$.

- Cible :

$$y_{i,t} = \ln \left(\frac{P_{i,t+h}}{P_{i,t}} \right),$$

- Prédiction du réseau :

$$\hat{y}_{i,t} = f_{\theta}(x_{i,t}).$$

On cherche θ qui minimise une fonction de perte sur l'échantillon d'apprentissage. Typiquement :

- MSE (Mean Squared Error) :

$$L_{\text{MSE}}(\theta) = \frac{1}{n} \sum_{(i,t)} (y_{i,t} - \hat{y}_{i,t})^2$$

2.2.2.3 Régularisation et optimisation

Le modèle peut facilement sur-apprendre (overfitting) si on fait un réseau trop large par rapport à la quantité de données. Pour limiter ça, on ajoute une régularisation L2 sur les poids :

$$L_{\text{total}}(\theta) = L_{\text{données}}(\theta) + \lambda \sum_{\ell} \| W^{(\ell)} \|_2^2,$$

où $\lambda > 0$ contrôle la pénalisation des grands poids.

L'optimisation se fait par descente de gradient stochastique (SGD) ou, plus souvent, par un optimiseur adaptatif type Adam :

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} L_{\text{total}}(\theta_t),$$

avec η le taux d'apprentissage. En pratique, on travaille en mini-batch, on utilise early stopping (on arrête l'entraînement quand la perte sur un set de validation ne s'améliore plus) et éventuellement une réduction automatique du taux d'apprentissage quand la perte stagne.

2.2.3 Données financières et construction du jeu d'apprentissage

2.2.3.1 Univers d'actions et horizon de prévision

On commence par choisir un univers d'actions. Pour chaque action i , on dispose d'un historique de prix $P_{i,t}$.

On fixe ensuite un horizon de prévision h (par exemple 5 jours si on réalloue chaque semaine). Pour chaque couple (action i , date t), on définit la cible :

$$y_{i,t} = r_{i,t+h} = \ln \left(\frac{P_{i,t+h}}{P_{i,t}} \right).$$

2.2.3.2 Variables explicatives (features) par action

Pour chaque action i et date t , on construit un vecteur de features $x_{i,t}$. On fait attention à ne jamais utiliser d'information future.

Exemples de features pertinentes :

- Rendements passés : $r_{i,t-1}, r_{i,t-2}, \dots, r_{i,t-k}$ (lags de rendement),
- Volatilité : écarts-types glissants sur 10, 20 ou 60 jours,
- Tendance : moyennes mobiles (5, 20, 60 jours), ratio prix / moyenne mobile,
- Indicateurs techniques simples : RSI, pourcentage de jours en hausse sur les 20 derniers jours
- Variables de marché global : rendement récent de l'indice, VIX, etc.,
- Variables cross-sectionnelles : secteur (encodé en one-hot), log de la capitalisation boursière, éventuellement un bêta estimé sur une fenêtre passée.

Ces features sont toutes causales : elles sont calculées à partir des informations disponibles au plus tard à t .

2.2.3.3 Format du dataset

Le dataset d'entraînement est un ensemble de paires $(x_{i,t}, y_{i,t})$ pour tous les couples (action, date) pour lesquels on a suffisamment d'historique pour construire les features et la cible. On empile tout ça dans une grande matrice :

- X : matrice de dimension (nombre d'observations) $\times$ (nombre de features),
- y : vecteur des log-rendements futurs.

On standardise ensuite chaque feature (soustraction de la moyenne, division par l'écart-type) en apprenant les statistiques uniquement sur le train pour éviter toute fuite d'information.

2.2.4 De la prédiction individuelle à l'allocation de portefeuille

2.2.4.1 Empiler les prédictions par date

Une fois l'ANN entraîné, pour une date de test t , on est capable de produire, pour chaque action i , une prédiction de rendement futur $\hat{r}_{i,t+h} = f_{\theta}(x_{i,t})$.

Pour la date t , on regroupe ces prédictions dans un vecteur :

$$\hat{\mu}_t = \begin{pmatrix} \hat{r}_{1,t+h} \\ \hat{r}_{2,t+h} \\ \vdots \\ \hat{r}_{N,t+h} \end{pmatrix}$$

où N est le nombre d'actions dans l'univers.

C'est ce vecteur $\hat{\mu}_t$ qu'on va transformer en vecteur de poids $w_t = (w_{1,t}, \dots, w_{N,t})^T$.

2.2.4.2 Règle d'allocation par softmax

On veut des poids qui vérifient :

- $w_{i,t} \geq 0$ (portefeuille long-only),
- $\sum_{i=1}^N w_{i,t} = 1$ (100 % investi),
- plus de poids aux actions avec meilleur rendement prédit.

Une règle simple consiste à :

1. Centrer et normaliser les prédictions à la date t :

moyenne des prédictions :

$$\bar{\hat{r}}_t = \frac{1}{N} \sum_{i=1}^N \hat{r}_{i,t+h}$$

écart-type :

$$\hat{\sigma}_{\hat{r},t} = \text{écart-type des } \hat{r}_{i,t+h}$$

score normalisé :

$$s_{i,t} = \frac{\hat{r}_{i,t+h} - \bar{\hat{r}}_t}{\hat{\sigma}_{\hat{r},t} + \varepsilon}.$$

2. Appliquer une fonction softmax :

$$w_{i,t} = \frac{\exp(\alpha s_{i,t})}{\sum_{j=1}^N \exp(\alpha s_{j,t})}.$$

Ici, $\alpha > 0$ est un paramètre :

- si α est petit, les poids restent proches d'un égal-pondéré ;
- si α est grand, le portefeuille se concentre fortement sur les quelques actions avec les plus gros scores.

Cette règle a l'avantage d'être :

- simple à coder,
- facile à expliquer,
- compatible avec un cadre long-only $\sum w_i = 1$.

2.2.4.3 Rendement du portefeuille et PnL après coûts

Une fois les poids w_t définis, le rendement du portefeuille entre t et $t + 1$ est :

$$R_{p,t+1} = \sum_{i=1}^N w_{i,t} r_{i,t+1}.$$

Pour prendre en compte les coûts de transaction, on introduit le turnover :

$$TO_t = \sum_{i=1}^N |w_{i,t} - w_{i,t-1}|.$$

Avec un coût proportionnel c (par exemple 0,1 % par unité de turnover), le PnL net est :

$$PnL_{t+1} = R_{p,t+1} - c \cdot TO_t.$$

C'est cette série de PnL qui sert à calculer :

- le Sharpe,
- le drawdown maximum,
- la volatilité,
- et à comparer la stratégie ANN à des benchmarks.

2.2.5 Protocole d'entraînement et validation (walk-forward)

2.2.5.1 Normalisation et fuites d'information

On standardise les features en appliquant :

$$x_j^{\text{norm}} = \frac{x_j - \mu_j}{\sigma_j},$$

mais avec une règle stricte :

- μ_j et σ_j sont calculés uniquement sur le jeu d'entraînement d'un bloc temporel donné (fold),
- on applique ces mêmes paramètres aux validations et au test pour ce bloc.

C'est une règle fondamentale en finance : on n'utilise jamais d'information du futur (même de manière indirecte).

2.2.5.2 Découpage walk-forward

Au lieu de faire un simple train/test fixe, on découpe la période d'étude en plusieurs blocs successifs :

- par exemple 4 à 6 folds ;
- pour chaque fold, on définit un segment d'entraînement, un segment de validation et un segment de test, en respectant l'ordre du temps.

Pour chaque fold :

1. On fit le scaler et le modèle ANN uniquement sur la partie TRAIN.
2. On utilise le set de validation pour tuner un peu les hyperparamètres (taille des couches, dropout...) et appliquer l'early stopping.
3. On prédit sur le TEST, on calcule les métriques ML (MAE/RMSE) et les métriques portefeuille (Sharpe, drawdown, etc.).
4. On passe au fold suivant en avançant dans le temps.

À la fin, on agrège les résultats sur tous les folds pour avoir une vision plus robuste.

2.2.6 Implémentation Python : ANN + allocation

2.2.6.1 Architecture ANN typique

En Keras (TensorFlow), un MLP adapté peut ressembler à :

- entrée : dimension = nombre de features,
- couches cachées : 2 ou 3 couches denses (ex. 128 → 64 → 32), activation ReLU, dropout 0,2,
- sortie : 1 neurone linéaire (rendement prédit).

2.2.6.2 Pipeline d'entraînement

1. Charger les prix, calculer les rendements, construire les features par action/date.
2. Construire le dataset (X, y) en empilant toutes les observations.
3. Découper en folds temporels (walk-forward).
4. Pour chaque fold :
fit du scaler sur X_{train} ,

entraînement de l'ANN avec early stopping,
prédictions sur la portion de test,
reconstruction des prédictions par date et par action,
allocation de portefeuille à partir de ces prédictions,
calcul des métriques de portefeuille.

2.2.6.3 Allocation et backtest

La conversion "prédictions $\rightarrow$ poids $\rightarrow$ PnL" suit la logique :

- à chaque date t :

on prend le vecteur de prédictions $\hat{\mu}_t$,

on calcule les scores normalisés $s_{i,t}$,

on applique la softmax pour obtenir $w_{i,t}$,

- on calcule le rendement de portefeuille $R_{p,t+1}$,
- on enlève un coût proportionnel au turnover pour obtenir le PnL net.

Ce pipeline peut être codé proprement dans un module dédié (allocation.py) pour bien séparer le rôle de l'ANN (prédire) et celui de la finance (allouer).

2.2.7 Limites de l'ANN et intérêt pour la suite du projet

Enfin, on peut conclure sur la place de l'ANN dans l'ensemble du projet :

- L'ANN est un modèle de base (baseline) sur données tabulaires : il ne voit pas la séquence brute comme un LSTM, mais il exploite très bien des features préparées (lags, vol, facteurs).
- Il fournit un premier vecteur de rendements attendus qui permet de tester la logique d'allocation (softmax ou Markowitz).
- On peut ensuite comparer ses performances à celles de modèles séquentiels plus sophistiqués (CNN, LSTM, GRU, Attention) en gardant le même pipeline d'allocation.

Les limites principales sont :

- sensibilité à la qualité des features (garbage in, garbage out),
- risque de sur-apprentissage si le réseau est trop gros,
- dépendance forte aux conditions de marché (non-stationnarité).

Mais justement, le fait d'avoir une brique ANN bien définie, proprement évaluée, donne une base solide pour montrer :

- ce que l'ANN sait faire,
- ce qu'il ne sait pas faire,
- et en quoi les architectures plus complexes (CNN/LSTM) apportent réellement quelque chose dans le contexte de séries temporelles financières.

2.3. Partie CNN

2.3.1 Motivation et positionnement des CNN dans le projet

Les réseaux de neurones convolutifs (Convolutional Neural Networks, CNN) sont historiquement associés au traitement d'images. Toutefois, leur principe fondamental, qui consiste à détecter automatiquement des motifs locaux dans des données structurées, s'applique naturellement au cas des séries temporelles financières. Dans ce projet, les CNN sont utilisés non pas comme des modèles de vision, mais comme des outils de traitement du signal appliqués à des séquences temporelles.

Contrairement aux réseaux de neurones artificiels classiques (ANN), qui exploitent des vecteurs de caractéristiques construits manuellement et ne modélisent pas explicitement la structure temporelle, les CNN opèrent directement sur des fenêtres temporelles ordonnées. À la différence des réseaux récurrents (RNN, LSTM), ils ne reposent pas sur une mémoire interne séquentielle, mais sur l'extraction de motifs locaux par convolution. Cette approche présente un intérêt particulier en finance, où de nombreux signaux sont de nature locale et répétitive, comme certaines configurations de rendements, de volatilité ou de ruptures de tendance.

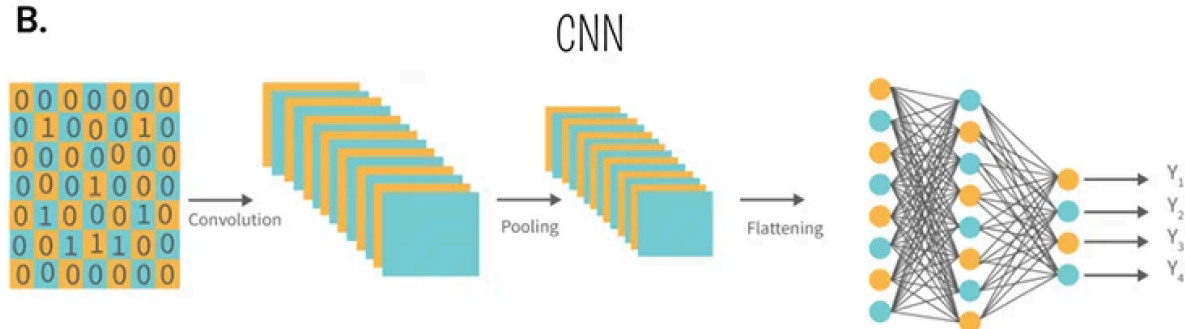
Les CNN constituent ainsi un modèle intermédiaire entre les ANN et les architectures récurrentes. Ils permettent de capturer des dépendances temporelles de court à moyen terme, tout en restant plus simples à entraîner et plus stables numériquement que les modèles récurrents profonds. Dans le cadre de ce projet, ils jouent un rôle clé pour analyser dans quelle mesure des motifs temporels locaux suffisent à produire des prédictions exploitables pour la construction de portefeuilles.

L'utilisation des CNN peut se résumer à :

- Entrées : Données financières (prix, volatilité, volumes, etc.) sont représentées sous forme de matrices numériques,
- Convolution : Application de filtres qui parcourent la matrice temporelle pour détecter des motifs locaux et ainsi remarquer les motifs les plus pertinents (création de carte caractéristique)

- Pooling : Réduction de la dimension ce qui permet de conserver l'information la plus importante tout en réduisant la complexité (limitant donc le surapprentissage)
- Flattening : Transformation des cartes de caractéristiques de deux voire trois dimensions en vecteurs à une dimension.
- Prédiction : Mise en relation des couches pour combiner les informations extraites par les convolutions pour apprendre des relations complexes entre motifs.
- Sorties : Prédiction d'un signal lié à la performance future de chaque actif de l'univers.

B.



Contrairement à l'ANN, qui traite des observations indépendantes $(x_{i,t} ; y_{i,t})$ le CNN exploite explicitement l'ordre temporel des observations et apprend automatiquement des motifs locaux dans le temps. Cette différence de biais inductif est centrale dans la comparaison des modèles.

2.3.2 Principes mathématiques du CNN

3.3.2.1 La convolution

Le cœur d'un réseau de neurones convolutif (CNN) appliqué aux séries temporelles financières repose sur l'opération de convolution unidimensionnelle. Nous allons maintenant essayer d'expliquer le principe de cette opération mathématique.

Dans sa forme la plus générale, la convolution est une opération définie entre deux fonctions réelles, permettant de produire une nouvelle fonction qui combine l'information contenue dans les deux fonctions initiales. Intuitivement, la convolution consiste à agréger les valeurs passées d'un signal en leur appliquant une pondération dépendant de leur distance temporelle.

Dans le cadre de ce projet, le signal d'entrée correspond à une série temporelle financière discrète, par exemple une suite de rendements ou de prix d'un actif financier, observée à intervalles réguliers. Le second terme de la convolution est un filtre,

également appelé noyau, qui contient un ensemble de coefficients appris par le réseau de neurones.

La convolution entre un signal $x(t)$ et un noyau $w(t)$ est définie par :

$$s(t) = (x * w)(t) = \int_{-\infty}^{+\infty} x(a) w(t - a) da$$

Cette expression peut être interprétée comme une moyenne pondérée du passé, où les observations $x(a)$ sont pondérées par une fonction w dépendant de leur ancienneté. D'un point de vue financier, cette opération est analogue à un indicateur de type moyenne mobile pondérée, mais avec une pondération apprise automatiquement plutôt que définie a priori.

En pratique, les données financières sont observées à des instants discrets (par exemple quotidiennement). On travaille donc avec une version discrète de la convolution.

Soit une série temporelle financière discrète :

$$x = (x_1, x_2, \dots, x_T)$$

Et un filtre de convolution de taille m :

$$w = (w_0, w_1, \dots, w_{m-1})$$

La convolution discrète unidimensionnelle est définie par :

$$s_t = \sum_{k=0}^{m-1} x_{t-k} w_k$$

Cette opération est effectuée pour chaque instant t tel que la fenêtre temporelle soit définie. Chaque valeur s_t résulte donc d'une combinaison linéaire locale des observations passées.

Dans les bibliothèques de deep learning, cette opération correspond en réalité à une corrélation croisée (sans retournement du noyau), mais cette distinction n'a pas d'impact fonctionnel dans un cadre d'apprentissage, puisque les poids du filtre sont appris directement par optimisation.

Dans un cadre réaliste, les séries financières sont multivariées. À chaque date t , on observe un vecteur de variables financières :

$$x_t \in \mathbb{R}^d$$

où d peut représenter, par exemple, le rendement, la volatilité, le volume, ou plusieurs indicateurs techniques.

Le filtre convolutionnel devient alors un tenseur de dimension $m \times d$. La convolution s'écrit :

$$s_t = \sum_{k=0}^{m-1} \sum_{j=1}^d w_{k,j} \cdot x_{t-k,j} + b$$

où b est un biais appris. Cette expression correspond à un produit scalaire entre une sous-séquence temporelle multivariée et le filtre, ce qui permet au réseau de capter des interactions conjointes entre plusieurs variables financières sur une fenêtre de temps donnée.

La sortie linéaire s_t est ensuite transformée par une fonction d'activation non linéaire ϕ , généralement la fonction ReLU :

$$a_t = \phi(s_t) = \max(0, s_t)$$

Cette non-linéarité est essentielle : sans elle, l'empilement de couches convolutionnelles se réduirait à une transformation linéaire globale, incapable de modéliser la complexité et les régimes non linéaires des marchés financiers.

Chaque filtre convolutionnel peut être interprété comme un détecteur de motif temporel. Dans la pratique, plusieurs filtres distincts sont utilisés simultanément au sein d'une même couche convolutionnelle. Chaque filtre possède ses propres poids et apprend à détecter un motif différent.

Contrairement aux indicateurs techniques classiques, qui reposent sur des formules définies a priori, ces motifs ne sont pas imposés par le modélisateur, mais émergent directement de l'optimisation du réseau sur les données historiques.

D'un point de vue financier, cette opération peut être interprétée comme un mécanisme d'apprentissage automatique de schémas locaux de marché, tels que :

- une succession de rendements négatifs suivie d'un retournement,
- une phase d'accélération de la volatilité,
- une dynamique de momentum sur une courte période,
- ou encore une rupture locale de régime.

La convolution permet ainsi de capturer des structures locales invariantes dans le temps, indépendamment de leur position exacte dans la série, ce qui est particulièrement adapté à l'analyse des séries financières bruitées.

Enfin, le partage des poids inhérent à l'opération de convolution impose une forme d'invariance temporelle : un motif donné est reconnu de la même manière quelle que soit sa position dans la fenêtre d'observation. Cette propriété est particulièrement

adaptée aux séries financières, où une configuration locale donnée conserve généralement la même signification économique indépendamment de la date à laquelle elle apparaît. Ainsi l'axe temporel joue le rôle principal de la convolution, tandis que les différentes variables financières peuvent être interprétées comme des canaux d'information distincts. Cette structuration des données est cohérente avec celle utilisée pour les réseaux récurrents, ce qui garantit une comparabilité directe des modèles dans les phases de prédiction et de backtest.

3.3.2.2 Le pooling

Après l'application d'une convolution et d'une fonction d'activation non linéaire sur une série temporelle financière, le pooling est une opération qui résume localement les activations pour produire une représentation plus compacte et robuste.

Soit a_t la sortie activée d'une convolution à l'instant t . Une opération de pooling remplace chaque sous-séquence $(a_t, a_{t+1}, \dots, a_{t+k-1})$ par un résumé statistique de cette fenêtre de taille k . Les fonctions de pooling les plus courantes sont :

1. Max pooling :

$$p_t = \max(a_t, a_{t+1}, \dots, a_{t+k-1})$$

2. Average pooling :

$$p_t = \frac{1}{k} \sum_{i=0}^{k-1} a_{t+i}$$

3. L2 pooling :

$$p_t = \sqrt{\sum_{i=0}^{k-1} a_{t+i}^2}$$

Ici, p_t est la sortie du pooling à l'instant t . Dans le cadre multivarié, le pooling s'applique indépendamment à chaque dimension, ou de manière combinée selon la structure du filtre.

On note quelques propriétés importantes de cette opération:

- Invariance locale aux translations : En regroupant les activations sur une fenêtre, le pooling rend la représentation insensible à de petites translations dans le temps. Par exemple, si un motif de retournement de marché se produit légèrement plus tôt ou plus tard que prévu, le pooling permettra toujours de le détecter.
- Réduction de dimension et efficacité computationnelle : Le pooling réduit le nombre de points de la série à traiter dans les couches suivantes. Si l'on prend

un stride s entre les fenêtres de pooling, le nombre d'éléments passe de T à $\approx T/s$. Cela diminue :

- le coût de calcul des couches suivantes,
 - le nombre de paramètres si la couche suivante est entièrement connectée,
 - la variance statistique des activations, ce qui améliore la généralisation.
- Robustesse au bruit : Dans les séries financières, les observations peuvent contenir du bruit aléatoire (erreurs de cotation, microstructure, fluctuations journalières). Le pooling, notamment le max pooling ou le L2 pooling, permet de filtrer ces fluctuations locales, ne conservant que les signaux significatifs détectés par la convolution.

Chaque fenêtre de pooling peut être vue comme un résumé local de l'état du marché. Par exemple, dans une série multivariée $x_t \in \mathbb{R}^d$:

- Un max pooling peut retenir la variable ayant connu la plus forte variation (ex : rendement extrême ou pic de volatilité) sur la fenêtre,
- Un average pooling peut capter la tendance générale ou la moyenne du momentum,
- Un L2 pooling peut quantifier l'intensité globale de la variation dans la fenêtre.

Ainsi, le pooling ne cherche pas à conserver la position exacte du signal, mais à extraire la présence et l'intensité d'un motif, ce qui est souvent suffisant pour des stratégies de prédiction ou de détection de régimes financiers.

Dans tous les cas, le pooling sert de pont entre la détection de motifs locaux (convolutions) et l'agrégation globale des informations pour les couches suivantes. Il permet de résumer efficacement l'information tout en réduisant la dimension des données, d'améliorer la robustesse face au bruit et de rendre le réseau partiellement insensible aux petites variations temporelles. Cette étape facilite ainsi l'apprentissage de représentations stables et significatives, essentielles pour la prédiction ou l'analyse de séries financières complexes.

2.3.3 Données et financières et représentation en entrée du CNN

Soit $P_{i,t}$ le prix de l'actif i à la date t . On définit le log-rendement élémentaire :

$$r_{i,t} = \log(P_{i,t}) - \log(P_{i,t-1})$$

Dans notre projet, l'information fournie au CNN ne se limite pas aux rendements. À chaque date t et pour chaque actif i , on construit un vecteur de variables financières observables :

$$\mathbf{x}_{i,t} = (x_{i,t}^{(1)}, x_{i,t}^{(2)}, \dots, x_{i,t}^{(d_x)}) \in \mathbb{R}^{d_x}.$$

Ces variables peuvent inclure notamment :

- des rendements passés à différents horizons,
- des mesures de volatilité glissante (10, 20, 60 jours),
- des informations de volume et de liquidité,
- des indicateurs de tendance simples (moyennes mobiles, ratios prix/moyenne mobile),
- des variables de marché globales communes à tous les actifs (indice de marché, volatilité implicite).

Toutes ces variables sont construites de manière strictement causale, sans utilisation d'information future.

Contrairement à l'ANN, l'entrée du CNN n'est pas un vecteur ponctuel $\mathbf{x}_{i,t}$, mais une fenêtre temporelle glissante de longueur T :

$$\mathbf{X}_{i,t} = (\mathbf{x}_{i,t-T+1}, \mathbf{x}_{i,t-T+2}, \dots, \mathbf{x}_{i,t}) \in \mathbb{R}^{T \times d_x}.$$

Cette matrice constitue l'entrée d'un CNN 1D. L'axe temporel (dimension T) est la dimension le long de laquelle s'applique la convolution, tandis que les d_x variables jouent le rôle de canaux d'information.

2.3.2 Limite des CNN pour la suite du projet

Les réseaux de neurones convolutifs peuvent, en théorie, être utilisés pour produire directement des poids de portefeuille en intégrant l'ensemble de la chaîne décisionnelle (de la prédiction des rendements jusqu'à l'allocation finale) au sein d'un unique modèle différentiable. Une telle approche end-to-end, dans laquelle le CNN apprend simultanément :

- une prévision implicite des rendements futurs,
- et une transformation de ces prévisions en allocations de portefeuille, présente une élégance théorique certaine.

Cependant, dans un cadre de gestion de portefeuille réaliste et institutionnel, cette approche soulève plusieurs difficultés majeures qui en limitent fortement la pertinence pratique.

Lorsque le CNN est entraîné pour produire directement des poids de portefeuille, il devient impossible de distinguer clairement :

- une erreur provenant de la brique de prévision (signal de rendement mal estimé),
- d'une erreur provenant de la règle d'allocation elle-même.

Cette confusion nuit fortement à l'analyse ex post des performances. Dans un cadre professionnel, il est essentiel de pouvoir diagnostiquer si une sous-performance provient d'un mauvais signal prédictif ou d'une mauvaise gestion du risque. Une architecture end-to-end rend cette distinction pratiquement impossible.

Les CNN end-to-end ont tendance à sur-apprendre non seulement les dynamiques de marché, mais également la règle d'allocation elle-même. Cela conduit fréquemment à:

- un turnover excessif, difficilement compatible avec des coûts de transaction réalistes,
- une sensibilité extrême aux changements de régime de marché,
- des performances très dégradées hors échantillon.

Cette instabilité est particulièrement problématique en finance, où la robustesse hors échantillon est un critère fondamental.

Dans la pratique institutionnelle, l'allocation de portefeuille est soumise à de nombreuses contraintes explicites :

- coûts de transaction et de liquidité,
- contraintes réglementaires,
- limites de concentration et de diversification,
- contraintes long-only ou bornes sur les poids.

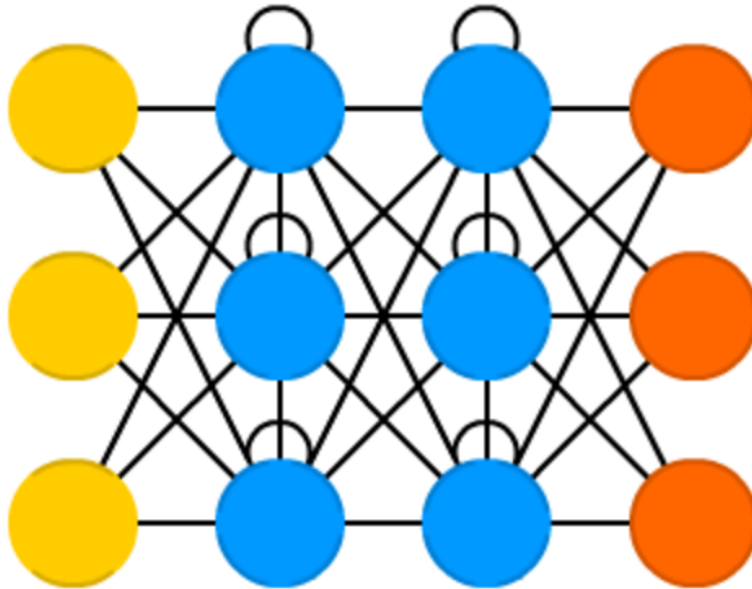
Intégrer ces contraintes directement dans une fonction de perte entraînant un CNN est possible en théorie, mais conduit à des fonctions objectives complexes, difficiles à calibrer et peu transparentes.

Enfin, une autre limite structurelle du CNN réside dans l'absence de mémoire explicite de long terme. Les dépendances temporelles capturées restent bornées par la taille de la fenêtre T et des filtres convolutifs.

Ces limitations justifient naturellement l'introduction de modèles récurrents, tels que les RNN et les LSTM, capables de capturer explicitement des dynamiques temporelles de long terme. Dans le cadre de ce projet, les CNN jouent ainsi un rôle intermédiaire et complémentaire, servant à la fois de modèle performant pour l'extraction de motifs locaux et de point de référence essentiel pour évaluer l'apport réel des architectures séquentielles plus complexes.

2.4. Réseaux récurrents (RNN) pour séries temporelles financières

Recurrent Neural Network (RNN)



Visualization of the Vanilla Recurrent Neural Network from Asimov Institute.

2.4.1 Motivation des RNN et Séries temporelles financières

Les réseaux de neurones récurrents (Recurrent Neural Networks, RNN) constituent une première classe de modèles spécifiquement conçus pour traiter des données séquentielles $(x_1, \dots, x_T)$, en introduisant une mémoire interne qui évolue au fil du temps. Contrairement aux modèles statiques, les RNN intègrent explicitement l'ordre temporel des observations et permettent de modéliser des dépendances dynamiques entre le passé et le présent.

Ce biais inductif les rend particulièrement adaptés à l'analyse des séries temporelles financières, qui se distinguent des jeux de données "classiques" par plusieurs caractéristiques structurantes. Elles sont ordonnées dans le temps, ce qui implique que l'information à la date t dépend directement de ce qui s'est produit aux dates $t - 1, t - 2, \dots$; Elles présentent également des dépendances de courte et de longue portée, telles que le momentum, le retour à la moyenne ou les régimes de volatilité. Enfin, elles sont fortement bruitées, souvent non stationnaires, et peuvent être entachées de données manquantes ou de fréquences irrégulières.

Dans un cadre standard de régression ou de classification, les modèles comme les régressions linéaires, les réseaux de neurones artificiels (ANN/MLP) ou les

approches tabulaires considèrent chaque observation (x_t, y_t) comme indépendante. La dépendance temporelle doit alors être introduite manuellement via des variables dérivées, telles que des retards de rendements, des moyennes mobiles, des volatilités glissantes ou des indicateurs techniques. Bien que cette approche reste pertinente dans certains contextes, elle demeure rigide et fortement dépendante du choix des features, ce qui limite sa capacité à capturer des dynamiques temporelles complexes.

Les RNN offrent une alternative conceptuelle en apprenant directement, à partir des données, quelles informations du passé doivent être conservées, atténuées ou oubliées, et comment cet historique influence une prédiction future, qu'il s'agisse d'un prix, d'un rendement ou d'une probabilité de changement de régime de marché. Ils constituent ainsi une première brique fondamentale pour la modélisation séquentielle en finance.

Toutefois, les RNN dits "vanilla" présentent des limitations importantes lorsqu'il s'agit de capturer des dépendances de long terme, en particulier dans le contexte des séries financières bruitées et non stationnaires. Ces limites ont motivé le développement d'architectures récurrentes plus avancées, telles que les LSTM (Long Short-Term Memory) et les GRU (Gated Recurrent Units), qui introduisent des mécanismes explicites de gestion de la mémoire. Les RNN standards restent néanmoins une base théorique indispensable, à partir de laquelle s'inscrit l'évolution vers ces modèles plus performants, au cœur de notre projet.

2.4.2 Prétraitement des séries temporelles financières pour les RNN

Avant d'entraîner un réseau récurrent (RNN, LSTM ou GRU), un prétraitement rigoureux des séries temporelles financières est indispensable. En pratique, nous suivons un pipeline standard largement documenté dans la littérature et dans les retours d'expérience industriels.

1. Gestion des données manquantes.

Les bases de données financières réelles présentent souvent des trous : jours fériés, suspensions de cotation, problèmes de flux de données, etc.

Dans ce projet, nous traitons ces valeurs manquantes en appliquant des méthodes simples mais robustes :

- Forward-fill (ffill) : chaque valeur manquante est remplacée par la dernière valeur observée ;
- Backward-fill (bfill) lorsque cela est pertinent, afin de ne pas créer de séries constantes trop longues ;
- éventuellement, une interpolation linéaire pour certaines grandeurs continues (indices, taux).

L'objectif n'est pas ici de "backtester" un modèle d'imputation, mais de reconstruire des séries utilisables par les modèles neuronaux.

2. Mise à l'échelle (normalisation / standardisation).

Les RNN et LSTM sont sensibles à l'échelle des variables d'entrée. Nous appliquons donc une standardisation :

$$x_{norm} = (x - \mu) / \sigma$$

où les paramètres (μ , σ) sont estimés uniquement sur le jeu d'entraînement d'un bloc temporel (walk-forward), puis réutilisés tels quels sur la validation et le test. Cette précaution évite les fuites d'information (data leakage) depuis le futur.

3. Travail sur les rendements plutôt que sur les prix.

Les niveaux de prix P_t sont en général non stationnaires. Nous travaillons donc le plus souvent sur les rendements logarithmiques :

$$r_t = \ln(P_t / P_{t-1})$$

éventuellement complétés par des indicateurs dérivés (volatilité glissante, indicateurs techniques simples, etc.).

4. Construction des séquences (fenêtres glissantes).

Enfin, nous transformons les séries en couples séquentiels $(X_{i,t}, y_{i,t})$ adaptés aux RNN :

$$X_{i,t} = (x_{i,t-T+1}, \dots, x_{i,t})$$

$$y_{i,t} = r_{i,t+h}$$

Où T est la longueur de la fenêtre d'historique et h l'horizon de prédiction. Le choix de T reflète un compromis entre la capacité à capturer des dépendances de plus long terme et la complexité du modèle.

Ce pipeline de prétraitement prépare les séries financières dans un format directement exploitable par les architectures récurrentes, tout en respectant la contrainte fondamentale de causalité (ne jamais utiliser d'information future pour prédire le passé).

2.4.3 Formulation d'un RNN "vanilla"

a) Notations et structure

On considère une séquence d'entrées $(x_1, \dots, x_T)$, où :

- $x_t \in R^{d_x}$ est le vecteur d'observations à l'instant t (par exemple : rendements, indicateurs techniques, facteurs de marché) ;
- $h_t \in R^{d_h}$ est le vecteur d'état caché (la mémoire interne du réseau à l'instant t) ;

• $\hat{y}_t \in R^{d_y}$ est la sortie prédite (par exemple un rendement futur ou une probabilité).

Un RNN “vanilla” met à jour l’état caché selon la récurrence :

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b_h)$$

$$\hat{y}_t = g(W_y h_t + b_y)$$

où :

- $W_x \in R^{d_h \times d_x}$: poids entre l’entrée et l’état caché ;
- $W_h \in R^{d_h \times d_h}$: poids de récurrence (de h_{t-1} vers h_t) ;
- $b_h \in R^{d_h}$: biais de la couche récurrente ;
- $W_y \in R^{d_y \times d_h}, b_y \in R^{d_y}$: poids et biais de la couche de sortie ;
- ϕ : fonction d’activation (souvent $\tanh$ ou ReLU) ;
- g : fonction d’activation de sortie (linéaire pour une régression de rendement, sigmoïde ou softmax pour une classification).

On initialise généralement :

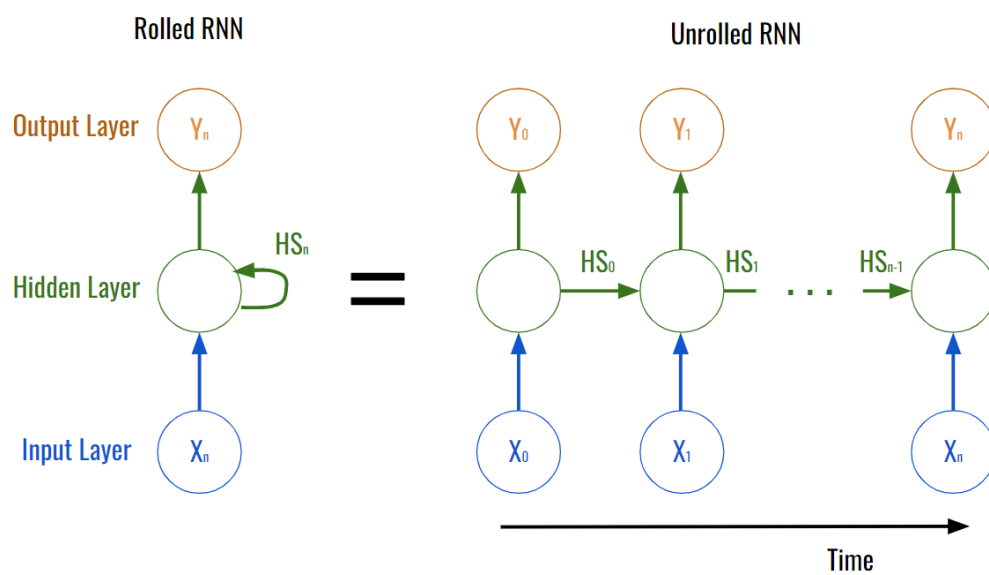
$h_0 = 0$ (ou un vecteur appris, ou une valeur aléatoire fixée).

b) Unrolling temporel

Pour visualiser un RNN, on “déplie” la récurrence dans le temps :

$$\begin{aligned}h_1 &= \phi(W_x x_1 + W_h h_0 + b_h), \\h_2 &= \phi(W_x x_2 + W_h h_1 + b_h), \\&\vdots \\h_T &= \phi(W_x x_T + W_h h_{T-1} + b_h).\end{aligned}$$

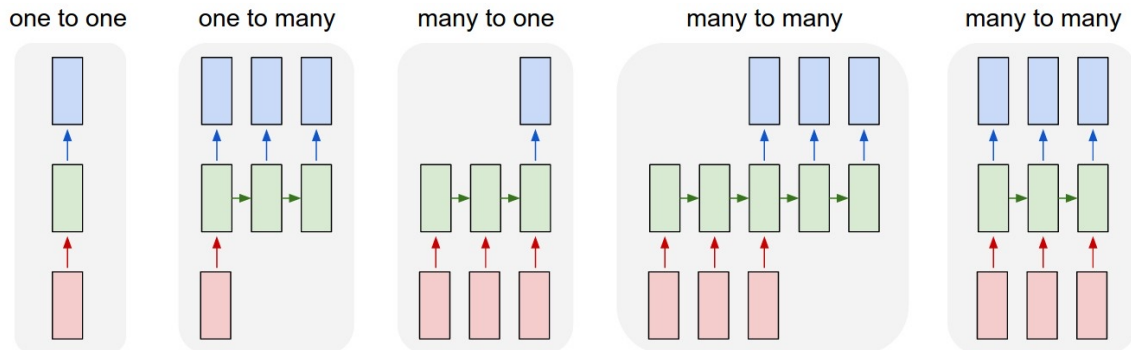
Graphiquement, on obtient un schéma où le même bloc “RNN” est répété pour $t = 1, \dots, T$, avec partage des poids W_x, W_h, b_h .



Rolled v.s Unrolled RNN

2.4.4 Différentes formes d'entrées/sorties (many-to-one, many-to-many)

Dans les applications financières, on utilise principalement deux types de schémas :



a) Many-to-one (séquence $\rightarrow$ prédiction unique)

On observe une fenêtre glissante de longueur T :

$$(x_{t-T+1}, \dots, x_t)$$

Et on souhaite prédire une quantité à horizon h (par exemple un log-rendement futur) :

$$y_{t+h} = f(x_{t-T+1}, \dots, x_t)$$

Dans ce cas :

- on passe la séquence à travers le RNN,
- on ne garde que l'état final h_t ,
- puis on applique une couche de sortie :

$$\widehat{y_{t+h}} = W_y h_t + b_y$$

C'est la configuration typique si l'on veut prédire le rendement ou le prix à horizon h à partir de l'historique récent.

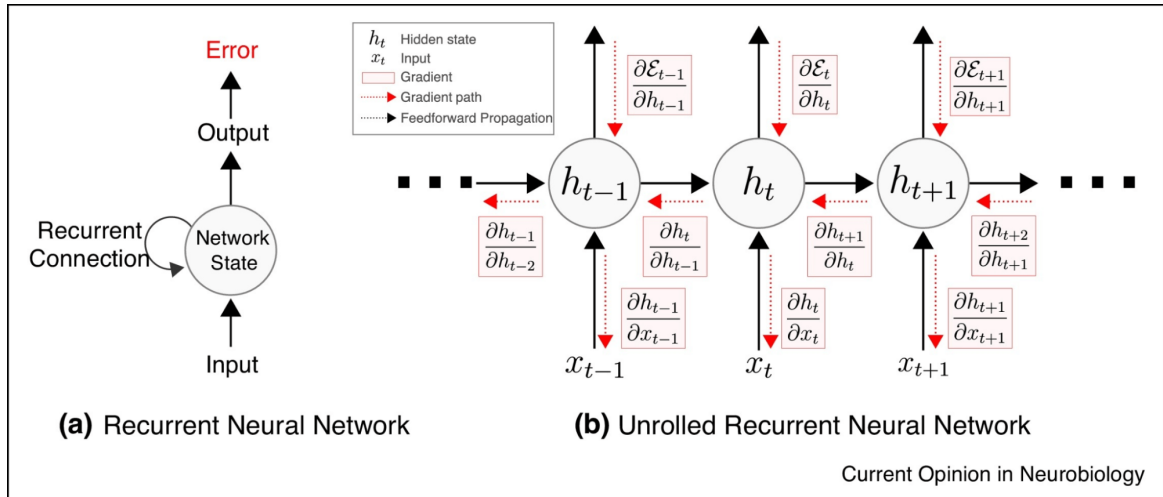
b) Many-to-many (séquence $\rightarrow$ séquence)

On peut aussi demander au RNN de produire une prédiction à chaque instant :

$$\hat{y}_t = g(W_y h_t + b_y), \quad t = 1, \dots, T$$

En finance, ce schéma peut servir à :

- prédire un rendement "one-step-ahead" à chaque date ;
- reconstruire des valeurs manquantes dans une série (imputation récurrente) ;
- produire une probabilité de "régime de marché" à chaque instant (classification séquentielle).



2.4.5 Fonction de perte et entraînement (BPTT)

a) Fonction de perte séquentielle

Pour une séquence d'entrée \$(x_1, \dots, x_T)\$ et de cibles \$(y_1, \dots, y_T)\$, la perte globale est souvent la somme des pertes instantanées :

$$\mathcal{L}(\theta) = \sum_{t=1}^T l(y_t, \hat{y}_t)$$

où \$l\$ est la perte élémentaire (par exemple MSE pour une régression de rendement) :

$$l(y_t, \hat{y}_t) = (y_t - \hat{y}_t)^2.$$

Dans le cas many-to-one (prédiction à l'horizon \$h\$), la perte peut se réduire à une seule date par séquence (la sortie finale).

b) Rétropropagation à travers le temps (Backpropagation Through Time – BPTT)

L'entraînement d'un RNN consiste à ajuster \$\theta = \{W_x, W_h, W_y, b_h, b_y\}\$ via un algorithme de descente de gradient (SGD, Adam, etc.).

La spécificité des RNN est que la rétropropagation doit tenir compte des dépendances temporelles :

$$\frac{\partial \mathcal{L}}{\partial \theta} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial h_t} \frac{\partial h_t}{\partial \theta}.$$

Or, \$h_t\$ dépend de \$h_{t-1}\$, qui dépend de \$h_{t-2}\$, etc. En développant, on obtient :

$$\frac{\partial \mathcal{L}}{\partial h_t} = \frac{\partial \mathcal{L}}{\partial h_T} \frac{\partial h_T}{\partial h_{T-1}} \dots \frac{\partial h_{t+1}}{\partial h_t}.$$

Les termes $\frac{\partial h_k}{\partial h_{k-1}}$ impliquent la dérivée de ϕ et la matrice W_h . On se retrouve alors avec des produits de matrices répétés.

C'est précisément ce mécanisme qui mène aux problèmes de gradient évanescent ou explosif.

2.4.6 Problème du gradient évanescent / explosif

a) Intuition mathématique

En simplifiant, supposons :

$$h_t = \phi(W_h h_{t-1}),$$

et que ϕ soit approximativement linéaire dans une région donnée. La dérivée en chaîne donne :

$$\frac{\partial h_T}{\partial h_t} \approx W_h^{T-t}.$$

Les valeurs propres de W_h jouent alors un rôle central :

- si $|\lambda_{\max}(W_h)| < 1$, alors $W_h^{T-t} \rightarrow 0$ lorsque $T - t$ augmente $\Rightarrow$ gradient évanescent ;
- si $|\lambda_{\max}(W_h)| > 1$, W_h^{T-t} croît exponentiellement $\Rightarrow$ gradient explosif.

Cela signifie que :

- le RNN standard oublie rapidement les informations lointaines (impossible de capturer des dépendances longues) ;
- ou au contraire devient numériquement instable.

b) Conséquences en finance

Sur des séries financières :

- les dépendances de court terme peuvent être capturées (effet d'un ou quelques jours) ;
- les régimes de marché longs, les cycles, la mémoire de volatilité sont mal appréhendés par un RNN vanilla.

C'est une limitation importante lorsqu'on cherche à modéliser, par exemple :

- l'impact d'une crise passée sur le comportement actuel ;
- la persistance de certaines tendances sur plusieurs semaines ou mois.

2.4.7 Limitations pratiques des RNN vanilla et motivation pour GRU / LSTM

Au-delà du problème théorique de gradient, les RNN standard présentent d'autres inconvénients :

1. Difficulté à entraîner sur des séquences longues

Même avec du gradient clipping, l'apprentissage reste fragile.

2. Absence de mécanisme explicite d'oubli

Le réseau doit apprendre implicitement à "oublier" certaines informations, ce qui le rend peu contrôlable.

3. Peu de capacités de "contrôle" sur la mémoire

Il est difficile d'interpréter directement ce que h_t mémorise. Ces limitations ont conduit au développement de RNN à portes (gated RNN), dont les plus connus sont :

- les GRU (Gated Recurrent Units).
- les LSTM (Long Short-Term Memory).

Ces architectures introduisent des portes différentiables (vecteurs sigmoïdes) qui contrôlent :

- quelle information est conservée ou oubliée ;
- quelle nouvelle information est ajoutée à la mémoire ;
- ce qui est exposé en sortie à chaque pas de temps.

2.4.8 Application typique d'un RNN vanilla à notre projet (pour comparaison)

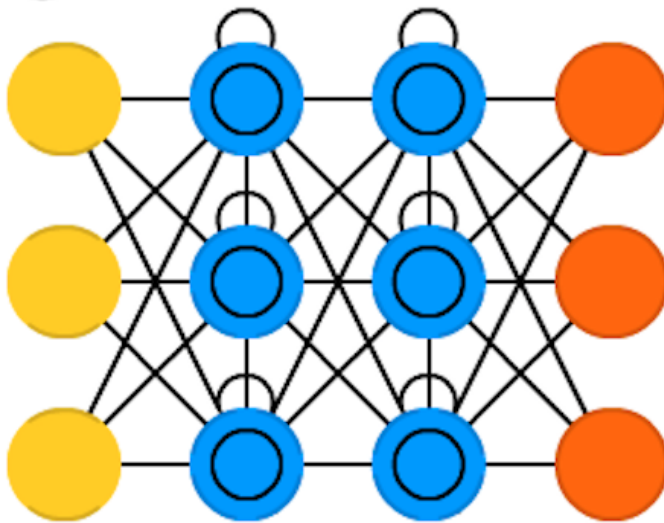
Même si notre projet s'appuie principalement sur des architectures plus avancées (LSTM, éventuellement GRU), il est utile de positionner le RNN vanilla comme baseline conceptuelle :

- Entrée : une fenêtre temporelle de rendements/indicateurs pour un actif : $(x_{t-T+1}, \dots, x_t)$
- RNN vanilla : mise à jour des états $h_{t-T+1}, \dots, h_t$;
- Sortie : prédiction d'un rendement futur $\widehat{r_{t+h}}$ à partir de h_t ;
- Perte : MSE entre r_{t+h} et $\widehat{r_{t+h}}$;
- Backtest : conversion des prédictions en poids de portefeuille, calcul des performances.

Dans la pratique, les résultats d'un RNN standard servent de référence pour :

- quantifier le gain apporté par l'introduction de portes (GRU, LSTM) ;
- illustrer l'effet positif d'une gestion explicite de la mémoire sur les performances de prédiction et d'allocation.

2.5. Long Short-Term Memory (LSTM)



Visualization of the Vanilla LSTM Network from Asimov Institute.

2.5.1 Origine et motivation du modèle LSTM

Les Long Short-Term Memory networks (LSTM) ont été introduits par Hochreiter et Schmidhuber (1997) afin de résoudre une limitation structurelle majeure des réseaux récurrents standards : l'incapacité à apprendre des dépendances temporelles de long terme en raison du phénomène de gradient évanescent.

Dans un RNN classique, l'information est propagée dans le temps via une succession de transformations non linéaires, ce qui entraîne une décroissance exponentielle (ou une explosion) des gradients lors de la rétropropagation. Ce problème est particulièrement critique pour les séries financières, dans lesquelles :

- certains effets persistent sur des horizons longs (volatilité, régimes de marché),
- les signaux utiles sont souvent noyés dans le bruit,
- l'impact d'événements passés peut se manifester avec retard.

Le LSTM introduit une mémoire explicite, séparée du calcul instantané, et contrôlée par des portes différentiables. Cette architecture permet au réseau d'apprendre quoi mémoriser, quoi oublier et quand utiliser l'information stockée, tout en conservant une stabilité numérique lors de l'apprentissage.

2.5.2 Structure générale d'une cellule LSTM

Contrairement à un RNN standard, une cellule LSTM repose sur deux états internes :

- un état *caché* $h_t \in R^{d_h}$, utilisé pour produire la sortie,
- un état mémoire (cell state) $c_t \in R^{d_h}$, destiné à transporter l'information sur de longues périodes.

L'évolution de ces états est régie par trois portes :

1. Forget gate : décide quelle partie de la mémoire passée doit être conservée,
2. Input gate : contrôle l'intégration de nouvelle information,
3. Output gate : détermine quelle partie de la mémoire est exposée en sortie.

Chaque porte est implémentée par une transformation affine suivie d'une fonction sigmoïde, ce qui garantit des coefficients compris entre 0 et 1.

2.5.3 Formulation mathématique complète du LSTM

On considère une séquence d'entrée $(x_1, \dots, x_T)$, a.é.

Les états internes du LSTM sont définis récursivement comme suit.

a) Forget gate

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

où :

- $f_t \in [0,1]^{d_h}$,
- $\sigma(\cdot)$ est la fonction sigmoïde,
- $W_f \in R^{d_h \times d_x}, U_f \in R^{d_h \times d_h}$.

Interprétation

Chaque composante de f_t agit comme un coefficient d'oubli.

- $f_t \approx 1$: l'information passée est conservée,
- $f_t \approx 0$: l'information passée est effacée.

En finance, cette porte permet par exemple au modèle d'oublier progressivement un choc ancien devenu non pertinent.

b) Input gate et mémoire candidate

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

- i_t contrôle l'intensité de l'écriture en mémoire,
- $\tilde{c}_t$ représente une mémoire candidate, issue de l'information courante.

c) Mise à jour de l'état mémoire

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Cette équation constitue le cœur du LSTM.

Point clé

La mise à jour de c_t est additive, et non multiplicative comme dans un RNN standard. Cette propriété est essentielle car elle permet aux gradients de circuler dans le temps sans s'annuler, ce qui garantit l'apprentissage de dépendances longues.

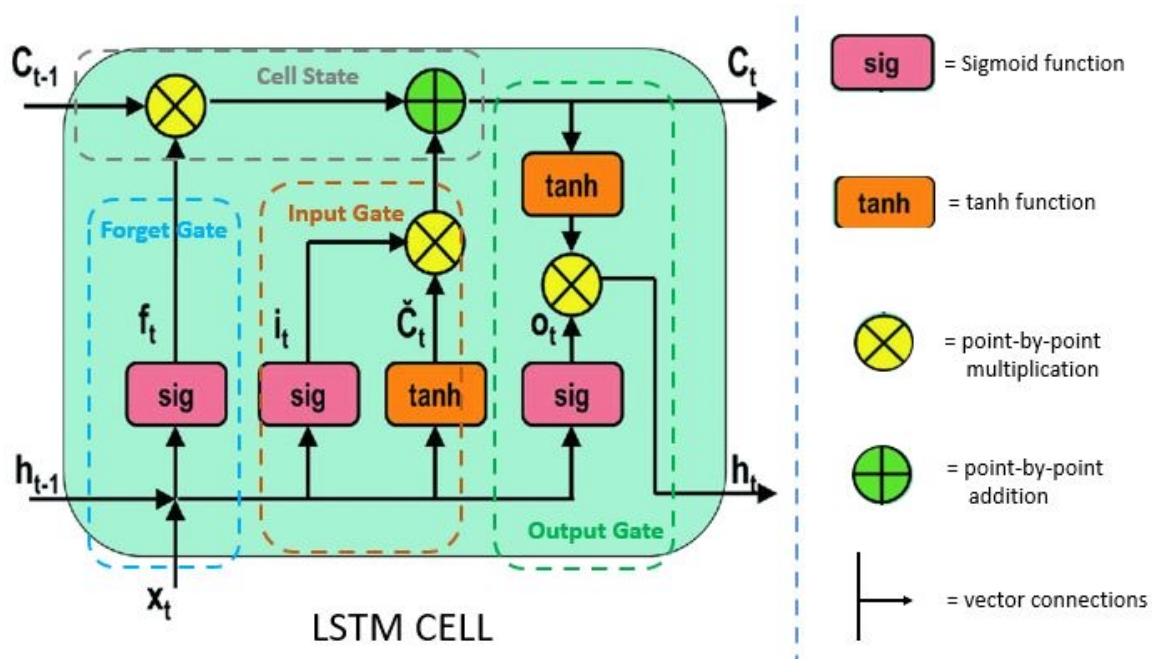
d) Output gate et état caché

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

$$h_t = o_t \odot \tanh(c_t)$$

L'état caché h_t est la partie de la mémoire interne rendue visible à l'extérieur du réseau, et utilisée pour produire la prédiction.

2.5.4 Schéma conceptuel de la cellule LSTM



2.5.5 Intuition dynamique et stabilité de l'apprentissage

Le LSTM peut être interprété comme un système dynamique contrôlé, dans lequel :

- la mémoire c_t joue le rôle d'un état interne lent,
- les portes agissent comme des vannes adaptatives,

- l'état caché h_t correspond à une projection instantanée de cette mémoire.

Grâce à cette architecture :

- les gradients se propagent principalement le long de c_t ,
- les dépendances longues peuvent être apprises,
- le réseau devient robuste à la longueur des séquences.

Ces propriétés expliquent pourquoi les LSTM sont largement utilisés dans des contextes où la structure temporelle est cruciale.

2.5.6 Interprétation financière du LSTM

Dans un contexte financier, l'état mémoire c_t peut capturer :

- des régimes de marché persistants (bull / bear),
- des phases prolongées de volatilité élevée ou faible,
- des dynamiques de tendance ou de retour à la moyenne.

Les portes permettent au modèle de :

- ignorer un choc ponctuel non structurel,
- conserver l'information d'une crise lorsque son impact se prolonge,
- adapter la mémoire aux changements de régime.

Ainsi, le LSTM agit comme un filtre temporel adaptatif, capable de distinguer le bruit de l'information structurelle.

2.5.7 LSTM pour la prédiction financière dans notre projet

Dans ce projet, le LSTM est utilisé pour :

- modéliser directement la séquence des rendements ou des prix,
- prédire une quantité future (rendement à horizon h),
- servir de modèle séquentiel de référence, comparé à un ANN basé sur des features.

Format des données

Pour chaque actif i , on construit des séquences glissantes :

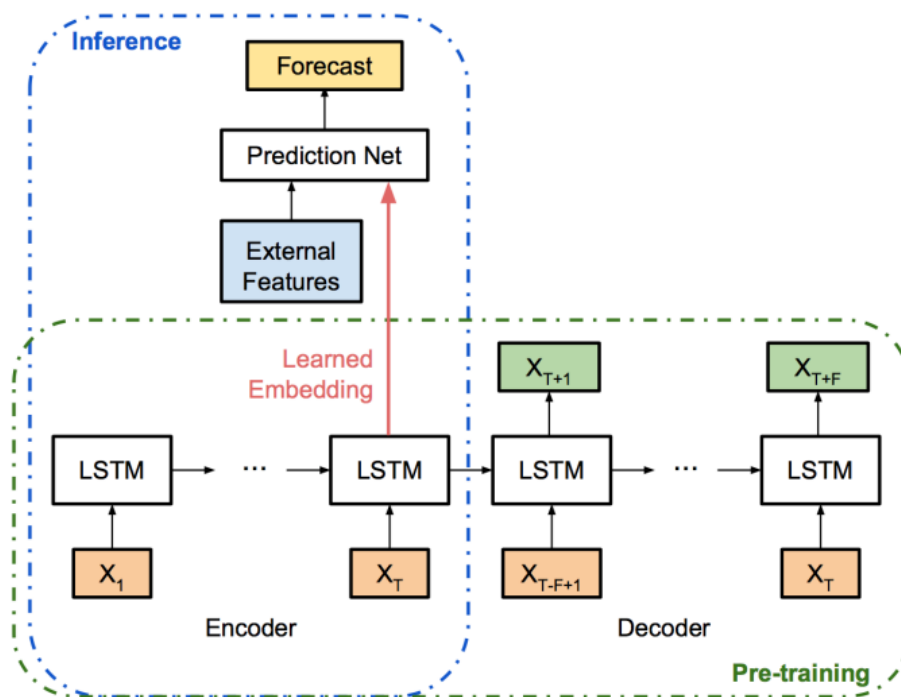
$$X_{i,t} = (x_{i,t-T+1}, \dots, x_{i,t})$$

avec comme cible :

$$y_{i,t} = r_{i,t+h}$$

Le LSTM apprend alors une fonction :

$$\widehat{y}_{i,t} = f_{\theta}(X_{i,t})$$



(Neural network architecture, with a pre-training phase using a LSTM encoder-decoder, followed by a prediction network, with input being the learned embedding concatenated with external features.

2.5.8 LSTM et allocation de portefeuille

Les prédictions du LSTM sont utilisées de la même manière que celles de l'ANN :

- agrégation des rendements prédits par actif,
- transformation en poids de portefeuille,
- backtest sur données historiques.

Cette cohérence de pipeline permet de comparer directement :

- la qualité prédictive,
- l'impact sur le rendement et le risque du portefeuille,
- la stabilité des allocations.

2.5.9 Limites du LSTM

Malgré ses avantages, le LSTM présente certaines limites :

- complexité plus élevée (temps de calcul, nombre de paramètres),
- sensibilité aux hyperparamètres (taille de la mémoire, longueur de séquence),
- difficulté d'interprétation économique fine des états internes,
- performances dépendantes de la qualité et de la quantité de données.

Ces limites justifient l'exploration de variantes ou d'architectures hybrides.

CF : CNN-LSTM (Jade wink wink)

Sources :

Nikolaj Buhl (2023) [Time Series Predictions](#)

Pik, J. (2020). Hands-On AI Trading with Python. Packt Publishing.

Lingxue Zhu, Nikolay Laptev (2017) [Deep and Confident Prediction for Time Series at Uber](#)

Mustapha Ez-Zaiym(2025) [Fractional Optimizers for LSTM Networks in Financial Time Series Forecasting](#)

Daksha Uchagaonkar [MEDIUM- LSTM- A Detailed Explanation](#)

Gavin Hull (2022) [Building a Neural Network Zoo From Scratch: The Recurrent Neural Network](#)

Fjodor van Veen(2016)[The Neural Network Zoo](#)

Fjodor van Veen(2017)[Neural Network Zoo Prequel: Cells and Layers](#)

3. DÉVELOPPEMENT ET INTERFACE UTILISATEUR

3.1. Objectif du site

- Application web pour un projet de Deep Learning appliqué à la finance.
- Deux fonctions principales :
 1. Prévion de courbes de prix pour plusieurs actifs financiers.
 2. Construction / visualisation d'une allocation de portefeuille à partir de ces actifs.

3.2. Structure générale de l'interface

- Page unique, divisée en :
Barre latérale gauche ($\approx 25\%$ largeur) : configuration et actions.
Zone principale droite ($\approx 75\%$ largeur) : affichage des résultats.
- Deux onglets dans la zone principale :
Onglet 1 : Visualisation des prévisions.
Onglet 2 : Allocation du portefeuille.

3.3. Barre latérale gauche (ruban)

3.3.1. Section « Importation des données »

Éléments :

- Bouton : « Importer un fichier CSV »

Permet à l'utilisateur de charger ses séries temporelles de marché et Traite les données manquantes.

- Liste déroulante : « Sélectionner le premier actif »

Affiche la liste des actifs disponibles.

Sélection multi-actifs :

- Une fois le « premier actif » sélectionné, un nouveau champ apparaît :

Liste déroulante : « Sélectionner le deuxième actif ».

- Une fois le deuxième actif sélectionné, apparition d'un troisième :

Liste déroulante : « Sélectionner le troisième actif ».

- Et ainsi de suite.
- Chaque nouvelle liste n'apparaît que si l'actif précédent a été choisi.

Sélecteurs de dates :

- Champ « Date de début »
- Champ « Date de fin »

- Définissent la période d'historique utilisée pour :
L'affichage des courbes.
L'entraînement des modèles.

3.3.2. Section « Paramètres d'entraînement »

Éléments :

- Champ numérique : « Taille de la fenêtre »
Longueur de la fenêtre glissante utilisée pour les séries temporelles (nombre de points d'historique en entrée du modèle).
- Champ numérique : « Nombre d'epochs »
Nombre d'itérations d'entraînement du modèle.
- Liste déroulante : « Modèle »
Valeurs : ANN, CNN, LSTM, GRU.
Permet de choisir la famille de réseau de neurones utilisée pour les prévisions.

3.3.3. Section « Prévvision future »

La section « Prévvision future » permet de contrôler l'horizon de projection du modèle au-delà de la dernière date observée. Elle comporte :

- Un champ numérique « Horizon de prévvision (en jours) », dans lequel l'utilisateur indique le nombre de jours qu'il souhaite prévvoir à partir de la dernière date de la série historique.
- Un bouton « Lancer la prévvision », qui déclenche la génération itérative des valeurs futures en utilisant le modèle déjà entraîné et les dernières observations disponibles.
Les courbes de prix sont alors prolongées dans l'onglet « Visualisation des prévvisions », la partie future étant distinguée visuellement de la partie historique.

3.3.4. Section « Actions »

Éléments :

- Bouton : « Générer les courbes »
Utilise les données importées + les dates + les actifs sélectionnés pour afficher les courbes historiques.
- Bouton : « Entraîner le modèle »
Lance l'entraînement du modèle choisi avec les paramètres (fenêtre, epochs).
Met à jour les prévvisions dans les onglets de droite.
- Bouton : « Télécharger les prévvisions »
Permet de récupérer les valeurs prédites (par exemple en CSV).

3.4. Zone principale droite

3.4.1. Navigation

- Barre d'onglets horizontale en haut de la zone principale :
 - Onglet 1 : « Visualisation des prévisions »
 - Onglet 2 : « Allocation du portefeuille »
- Un seul onglet visible à la fois.

3.4.2. Onglet 1 – « Visualisation des prévisions »

1. Graphique principal multi-actifs

- Axe des abscisses : temps (dates).
- Axe des ordonnées : prix ou niveau de l'actif.

Affiche :

- Courbe(s) historique(s) réelle(s).
- Courbe(s) prévue(s) par le modèle.

Chaque actif a une couleur dédiée.

Légende listant les actifs + type de série (réelle / prédite).

2. Visualiser la qualité des prévisions.

- Comparer rapidement les performances du modèle sur les différents actifs à l'aide d'indicateurs concernant la précision des prévisions.
- Sous le graphique principal, trois indicateurs quantitatifs sont affichés afin d'évaluer la qualité des prévisions du modèle :
 - MAE (Mean Absolute Error) : mesure l'erreur absolue moyenne entre les valeurs réelles et les valeurs prédites. Il indique, en unités de prix, l'erreur moyenne commise par le modèle.
 - RMSE (Root Mean Squared Error) : mesure la racine de l'erreur quadratique moyenne. Il pénalise plus fortement les grandes erreurs que le MAE et permet de détecter la présence de prévisions très éloignées des valeurs observées.
 - R^2 (coefficient de détermination) : mesure la proportion de la variabilité des prix expliquée par le modèle. Une valeur proche de 1 traduit un modèle explicatif, tandis qu'une valeur proche de 0 indique que le modèle explique peu la dynamique de la série.

Ces indicateurs sont calculés sur la période de test et mis à jour après chaque entraînement ou nouvelle prévision.

3.4.3. Onglet 2 – « Allocation du portefeuille »

Contenu :

1. Titre / section principale

Titre : « Allocation optimisée du portefeuille »

2. Paramètres d'allocation

Liste déroulante : « Horizon d'investissement »

- Options : Court terme / Moyen terme / Long terme.

Champ ou curseur : « Volatilité cible »

- Niveau de risque souhaité.

Champ ou curseur : « Rendement espéré »

- Objectif de performance.

3. Graphique d'allocation

Représentation des poids par actif :

- Soit camembert, soit barres verticales.

Chaque actif hérite de la même couleur que dans l'onglet 1 pour cohérence visuelle.

4. Résultats numériques

Tableau ou liste indiquant :

- Nom de chaque actif.
- Poids du portefeuille (en %).

Carte synthèse « Profil risque-rendement » :

- Volatilité estimée du portefeuille.
- Rendement estimé.

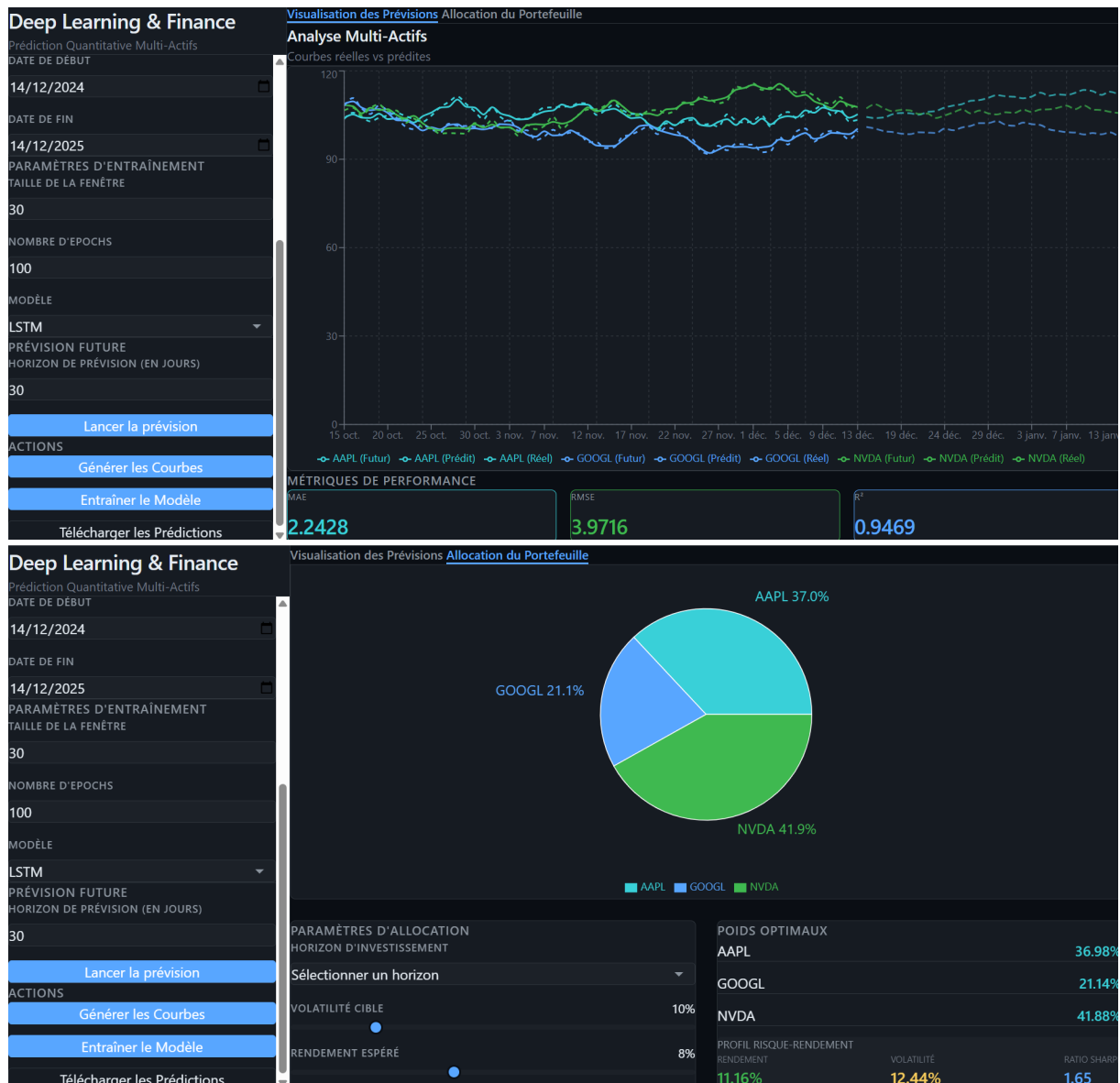
Objectif de cet onglet :

- Montrer comment les actifs sélectionnés peuvent être combinés en portefeuille.
- Faire le lien entre prévisions de prix et construction d'allocation.

3.5. Comportements / logiques de base

1. Les actifs affichés dans les deux onglets sont ceux sélectionnés dans la barre latérale
2. Les paramètres d'entraînement influencent :
 - Les courbes prédites dans l'onglet 1.
 - Les métriques de performance.
3. L'allocation de l'onglet 2 est calculée uniquement sur les actifs effectivement sélectionnés.

4. Le bouton « Télécharger les prédictions » exporte les séries prédites pour tous les actifs sélectionnés.



Biblio ANN

<https://www.sciencedirect.com/science/article/abs/pii/S0957417423019061>

<https://arxiv.org/abs/2009.03394>

<https://amslaurea.unibo.it/24245/>

<https://arxiv.org/html/2411.13555v3>

PLAN PREZ

(0s)

1- PHOTO STYLE

(1m)

2-INTRO / Prez du projet

(1m)

3- Problématique (Accuracy du modèle, utilisateur , taille du data-set)

(30s)

4- Fetching et Prep des données

(30s)

5- Différents type des modèles : ANN, CNN, récurrents

(4m)

6- Prez ANN, (Exemple), comment est-ce qu'il se distingue (mathématiquement et en vrai) + DEFAULTS (non-respect de nos attentes)

(4M)

7- Prez CNN, (Exemple), comment est-ce qu'il se distingue (mathématiquement et en vrai) (non-respect de nos attentes)

(4m)

8- Prez RNN-LSTM, Exemple, comment est-ce qu'il se distingue (mathématiquement et en vrai) (non-respect de nos attentes)

(2m)

9 Notre solution : MIX de CNN et LSTM , le meilleur des deux monde Explication embranchement mathématique.

(2-3m)

10- Solution et rendu graphique avec les fonctionnalités offertes à l'utilisation

Total : ±19m

Creation du por

Intro

-slide de couv du projet

1-prez projet (à quoi il sert, pour qui ? quelles fins ? comment on fait, par quoi on commence)

- Objectif du projet : *Application de modèles de réseaux de neurones pour la prédiction de cours de sous-jacents afin d'optimiser l'allocation d'actifs d'un portefeuille*
- *Actifs choisis*
- Réseaux de neurones
- À qui il s'adresse : particuliers,

2-Structure de base (recup des données, yahoo finance, csv, performant avec panda)(traitement des données récupérées (notamment valeurs manquantes) (input, output),

- Quelles données (vecteurs de données)
- standardiser
- Comment gerer les données manquantes (cours de ML)

3-presente les modèles (2-3 caractéristiques clés du modèle, rapide explication mathématique du modèle (bcp de schéma), tableau comparatif des modèles ==> dire celui qu'on choisi (CNN pipeline LSTM)

- donnée entrées
- spécificités mathématiques
- Sortie
- tableau comparatif → explication de notre choix de modèle final
- Allocation

4-Outil

- Dev de l'outil
- Liste des fonctionnalités (Cahier des charges)

5-ccl

- slide question
- remerciement